

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
CIÊNCIA DA COMPUTAÇÃO

ANDRÉ DELGADO HERNANDEZ
GERMANO GARLET DE BORTOLI

TRABALHO FINAL
RELATÓRIO - DONKEY KONG
ALGORITMOS E PROGRAMAÇÃO- INF01202

PORTO ALEGRE
2026

EXPLICAÇÃO DE COMO USAR O PROGRAMA

Para executar o programa, acesse a pasta *MeuJogo/* e abra o arquivo *MeuJogo.cbp* no Code::Blocks com a biblioteca *Raylib* já configurada no ambiente e clique em *Build and Run (F9)*. É necessário que todos os arquivos estejam dentro da pasta *MeuJogo/*: os arquivos de mapa (*mapa1.txt*, *mapa2.txt*, *mapa3.txt*), a pasta *audio/* com as músicas do jogo e a pasta *graphics/* com as sprites.

Conforme indicado na tela de menu, utilize os seguintes controles:

- Setas ← / → - movimentação horizontal
- Setas ↑ / ↓ — subir e descer escadas
- SPACE - pular
- TAB - pausar o jogo
- F1 - mudar para tela cheia
- ESC - fechar o jogo

ESPECIFICAÇÃO DE COMO OS ELEMENTOS DO JOGO FORAM REPRESENTADOS

Organizamos o jogo em um arquivo principal ‘jogo.c’ e dividimos grande parte das funções utilizadas em bibliotecas específicas, buscando facilitar o entendimento e desenvolvimento do código.

As bibliotecas criadas foram:

- ‘button.h’
- ‘constantes.h’
- ‘funcoes_desenha_jogo.h’
- ‘funcoes_fase.h’
- ‘funcoes_inimigos.h’
- ‘funcoes_jogador.h’
- ‘funcoes_mapa.h’
- ‘placar.h’

Em todas as bibliotecas, utilizamos os comandos *#ifndef*, *#define* e *#endif*, com o objetivo de garantir que em cada execução do jogo cada header só será incluído uma vez, pois há bibliotecas incluídas dentro de outras. Os comandos executam da seguinte forma: caso o header não tenha sido definido, defina-o e, ao final do código, finalize o comando condicional. As funções e estruturas utilizadas serão explicadas na sequência, conforme a biblioteca:

- I. button.h
 1. Definição da Struct “Botao”
 2. Constantes do posicionamento dos botões
- II. constantes.h
 1. Definição das dimensões da matriz do mapa
 2. Definição de TILE_SIZE (quantos pixels cada posição da matriz ocupa)
 3. Definição de MAX_INIMIGOS (número máximo de inimigos no jogo)
 4. Definição de FPS (frames per second)
- III. funcoes_desenha_jogo.h
 1. void desenharMapa: recebe a matriz do mapa e as sprites e desenha algo na posição x da tela dependendo do caractere da matriz (switch-case)
- IV. funcoes_fase.h
 1. int passouFase: recebe a linha em que o jogador se encontra e verifica se o jogador sobe uma linha. Se o jogador “sair para cima do mapa”, retorna 1, caso contrário, retorna 0.

2. `int voltouFase`: recebe a linha em que o jogador se encontra e verifica se o jogador desceu uma linha. Se o jogador “sair para baixo do mapa”, retorna 1, caso contrário, retorna 0.
 3. `int vitoria`: recebe a matriz do mapa e as posições x e y do jogador. Se a posição do jogador for igual ao caractere ‘F’, retorna 1, caso contrário, retorna 0.
- V. `funcoes_inimigos.h`
1. Definição da struct “INIMIGO”
 2. `void encontrarInimigos`: recebe a matriz do mapa, o vetor que armazena inimigos, e o ponteiro de total de inimigos. Define a posição inicial de todos os inimigos, armazena os inimigos no vetor e conta quantos são.
 3. `void movimentarInimigos`: recebe a matriz do mapa, o vetor que armazena inimigos, o valor do total de inimigos e um ponteiro contador. Só permite a movimentação dos inimigos em determinadas condições do mapa, e o contador define a velocidade.
 4. `int colisaoInimigo`: recebe a posição do jogador, o vetor de inimigos e o total de inimigos. Se a posição do jogador for igual a posição de algum dos inimigos do vetor, retorna 1, caso contrário, retorna 0.
 5. `void desenharInimigos`: recebe o vetor de inimigos, o total de inimigos e as sprites deles. Desenha os inimigos na tela de acordo com a posição de cada um deles.
- VI. `funcoes_mapa.h`
1. `void carregarMapa`: recebe uma matriz de mapa e um nome de arquivo. Abre o arquivo .txt e carrega os caracteres na matriz.
 2. `int temChao`: recebe um caractere. Verifica se o caractere é uma plataforma, borda, ou meio de escada.
- VII. `placar.h`
1. Definição da estrutura “TIPO_PLACAR”
 2. `void salvarResultadoNoPlacar`: recebe um ponteiro “nome” e o tempo jogado. Salva em um arquivo binário “placar.bin”
 3. `void limparPlacar`: não recebe argumento e limpa o arquivo “placar.bin”.
 4. `int deveEntrarNoRanking`: recebe o inteiro do tempo do jogador. Verifica se o tempo do jogador está entre o top10 do ranking.

Criamos uma pasta ‘audio’ destinada para os arquivos de som (.mp3) utilizados no jogo e uma pasta ‘graphics’, utilizada para armazenar as imagens (sprites) utilizadas no jogo. Desenvolvemos 3 mapas (‘mapa1.txt’, ‘mapa2.txt’ e ‘mapa3.txt’). O projeto do jogo ficou salvo no arquivo ‘MeuJogo.cbp’.

A base do nosso jogo foi implementada a partir de um typedef enum, definindo ‘EstadoJogo’, isto é, em que tela o jogo se encontra, para posteriormente ser implementado switch case com cada estado. Daí foram definidas 5 telas: ‘TELA_MENU’, ‘TELA_JOGO’, ‘TELA_PAUSA’, ‘TELA_RANKING’, ‘TELA_INPUT_NOME’. A partir disso, criamos dois *Switch Cases*, um para a parte lógica de atualização do jogo (implementado primeiro) e outro para o desenho (onde as funções `BeginDrawing()` e `EndDrawing()` atuam).

ESTRUTURAS UTILIZADAS

Foram definidas quatro *structs* ao longo do projeto:

- **PLAYER**: representa o jogador. Contém linha e coluna (posição na matriz do mapa), `velY` (velocidade vertical), `contadorGravidade` (contador de frames para controlar a frequência da gravidade) e `direcao` (valor 1 para direita e -1 para esquerda, usado para selecionar a sprite adequada (há duas, uma para direita e outra para esquerda)).

- INIMIGO: representa cada inimigo do jogo. Contém linha e coluna (posição na matriz do mapa) e direcao (1 para direita e -1 para esquerda).
- TIPO_PLACAR: representa uma entrada do ranking. Contém nome[20] (string para o nome do jogador) e time (tempo em segundos do jogo para pontuação). O arquivo *placar.bin* é formado por no máximo 10 structs desse tipo, mantendo ordenado do menor tempo para o maior (com *bubble sort*).
- Botao: representa um botão do menu. Contém rect (do tipo *Rectangle* da *Raylib*, define a área a ser clicada com o botão esquerdo do mouse) e texture (do tipo *Texture2D* da *Raylib*, a imagem exibida na área do botão).

EXPLICAÇÃO DO CÓDIGO (JOGO.C)

Incluímos todas as bibliotecas a serem utilizadas, além da *<stdio.h>* e *<stdlib.h>*. Definimos as variáveis do tipo *EstadoJogo*, indicando em qual tela o jogo está. Abrimos a função principal *int main(void)* e inicializamos a janela com tamanho 750x750 pixels, com o título “Donkey Kong - INF”.

Determinamos como 60 FPS a taxa de atualização da tela. Inicializamos a fonte utilizada na definição das teclas no menu. Carregamos todas as músicas salvas na pasta *audio/*, com a função *LoadMusicStream()*, e começamos a tocar a música do Menu. Carregamos todas as imagens salvas na pasta *graphics* com a função *LoadTexture()*. Após isso, inicializamos todas as variáveis com seus respectivos tipos a serem utilizadas durante o jogo. Declaramos os botões do menu, implementados a partir da *struct Botao*. E, por fim, definimos *telaAtual* (do tipo *EstadoJogo* para usar no *switch-case*) como *TELA_MENU*, isto é, para inicializar o jogo na tela do menu. Iniciamos o loop principal do jogo com o comando *while(!WindowShouldClose())*, que continua rodando até que a tecla *ESC* seja pressionada. Inicializamos a posição do mouse com o tipo *Vector2*, para utilização posterior (para verificar se o ponteiro está em cima dos botões para processar os cliques). Caso o jogador queira que o jogo fique em tela cheia, basta pressionar a tecla *F1*, que aplica a função *ToggleFullscreen()*.

Finalmente iniciamos o *switch-case* com o argumento *telaAtual* focado na lógica do jogo. Dentro do case *TELA_MENU*, há condicionais *if* para verificar se o usuário clicou com o botão esquerdo do mouse em cima da área do botão de ‘Novo Jogo’ e, dentro desse *if*, zeramos todas as variáveis (pois é uma nova partida) e utilizamos as funções para carregar o mapa, definir a posição do jogador e definir a posição dos inimigos, parar a música do menu e iniciar a música da Fase 1, além de mudar o *telaAtual* para *TELA_JOGO*. Em outro *if*, caso o jogador clique com o botão esquerdo do mouse em cima da área do botão de ‘Ranking’, muda a *telaAtual* para *TELA_RANKING*. Por último, caso o jogador clique em ‘Sair’, descarregamos todas as sprites e áudios utilizados e executamos o *return 0* para encerrar o programa.

No segundo case (*TELA_JOGO*), fazemos um controle do tempo dentro de um condicional, isto é, se o jogador não morreu, não venceu e não está em animação de troca de fase, incrementa o tempo de jogo. Caso o jogador pressione a tecla *TAB*, muda a *telaAtual* para *TELA_PAUSA* e dá um *break*. Caso *gameOver* seja verdadeiro (valor diferente de 0), incrementa o *timerGameOver* e, depois de 3 segundos, para de tocar quaisquer músicas que estejam ativas, atualiza *telaAtual* para *TELA_MENU* e começa a tocar a música do menu. Caso ganhou seja verdadeiro, incrementa o *timerGanhou* e, depois de 3 segundos, encerra as músicas, atualiza *telaAtual* para *TELA_INPUT_NOME*, reinicia a música do menu e inicializa *nomeJogador* com ‘\0’ e *letrasContadas* para 0, permitindo que o jogador insira seu nome caso tenha entrado no Top 10. Caso *trocandoFase* seja verdadeiro, aplicamos os comandos para mudar de fase e carregar o mapa respectivo, além de tocar a trilha sonora adequada. Além disso, recarregamos os inimigos, zeramos a gravidade, o contador de frames dos inimigos e desativamos a flag de troca de fase. Aplicamos as funções de movimentação tanto do jogador quanto dos inimigos. Aplicamos também um condicional

para verificar se o jogador colidiu com algum inimigo (ativando a flag *gameOver*), juntamente com outro condicional que verifica se o jogador alcançou o objetivo final (ativando a flag *ganhou*).

No terceiro *case* (*TELA_PAUSA*), se o jogador pressionar novamente *TAB*, retorna para *TELA_JOGO*; caso pressione *M*, retorna para *TELA_MENU*; e caso pressione *S*, sai do jogo e descarrega as sprites e áudios da memória RAM.

No quarto *case* (*TELA_RANKING*), garantimos que a música do menu está tocando. Caso a tecla *M* seja pressionada, retorna para *TELA_MENU*. A lógica de leitura do arquivo binário *placar.bin* é processada aqui para carregar o vetor de estruturas de ranking que será exibido.

No quinto *case* (*TELA_INPUT_NOME*), inicializamos uma variável inteira chave que recebe o retorno da função *GetCharPressed()*. Enquanto a chave for maior que 0, estiver dentro do intervalo visível da tabela ASCII (33 a 124) e o vetor contiver no máximo 19 caracteres, altera-se o vetor *nomeJogador[letrasContadas]*, adicionando um ‘\0’ para garantir o fechamento da string e incrementando *letrasContadas*. Caso a tecla *BACKSPACE* seja pressionada, *letrasContadas* é decrementado, possuindo uma validação para nunca ser menor que 0. Caso o usuário pressione *ENTER* (e o número de letras seja maior que 0), aplicamos a função *salvarResultadoNoPlacar*, paramos a música existente e atualizamos a tela para *TELA_RANKING*. Fora dos *cases*, aplicamos a função *UpdateMusicStream()* para manter o fluxo de áudio da *Raylib* ativo a cada frame.

Finalmente, iniciamos a parte do código responsável por desenhar os elementos visuais a partir de um segundo switch-case. No *case* *TELA_MENU*, desenhamos os textos dos controles e os botões, aplicando o efeito de hover (mudança de cor) caso o mouse passe por cima da área delimitada.

No segundo *case* (*TELA_JOGO*), limpamos a tela com um fundo preto e desenhamos o mapa, o jogador e os inimigos. Inserimos o texto indicador da fase atual e o tempo de partida. Colocamos 3 condicionais visuais: o primeiro desenha “...” quando está trocando de fase, o segundo desenha “GAME OVER” caso o jogador perca, e o terceiro desenha “VITÓRIA” caso vença.

No terceiro *case* (*TELA_PAUSA*), desenhamos o texto “PAUSADO” e as instruções de comandos (*TAB* para continuar, *M* para o menu e *S* para encerrar).

No quarto *case* (*TELA_RANKING*), desenhamos o título “RANKING”. Utilizando um laço *for* baseado na quantidade de jogadores existentes no arquivo, usamos o comando *sprintf* para formatar o nome e a pontuação dentro de uma string temporária, desenhando linha por linha com a função *DrawText* (ajustando a posição *Y* dinamicamente). Adicionamos as opções visuais de pressionar *M* para voltar ao menu e *L* para limpar o ranking (que chama a função *limparPlacar()*).

No quinto *case* (*TELA_INPUT_NOME*), desenhamos o fundo preto, o texto “VITÓRIA! INSIRA SEU NOME:” e a caixa de entrada de texto. Para que o usuário veja seu nome em tempo real, passamos a variável *nomeJogador* dinamicamente na função *DrawText*. Por fim, desenhamos “Pressione *ENTER* para salvar”. Ao quebrar o loop principal, descarregamos todas as imagens e áudios restantes da memória RAM, fechamos a janela do jogo e finalizamos com *return 0*;

SPRITES UTILIZADAS





ESTRUTURA



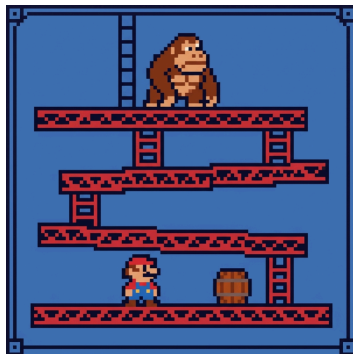
BOTÃO NOVO JOGO



BOTÃO RANKING



BOTÃO SAIR



CAPA DO MENU